

# Explainable AI: Concepts, Techniques, and Implementations

## From Taxonomy to Practical Methods (LIME, SHAP, Grad-CAM, and Beyond)

# Previous session

- SHAP & LIME with mathematical foundation and numerical example

NIPTEEL

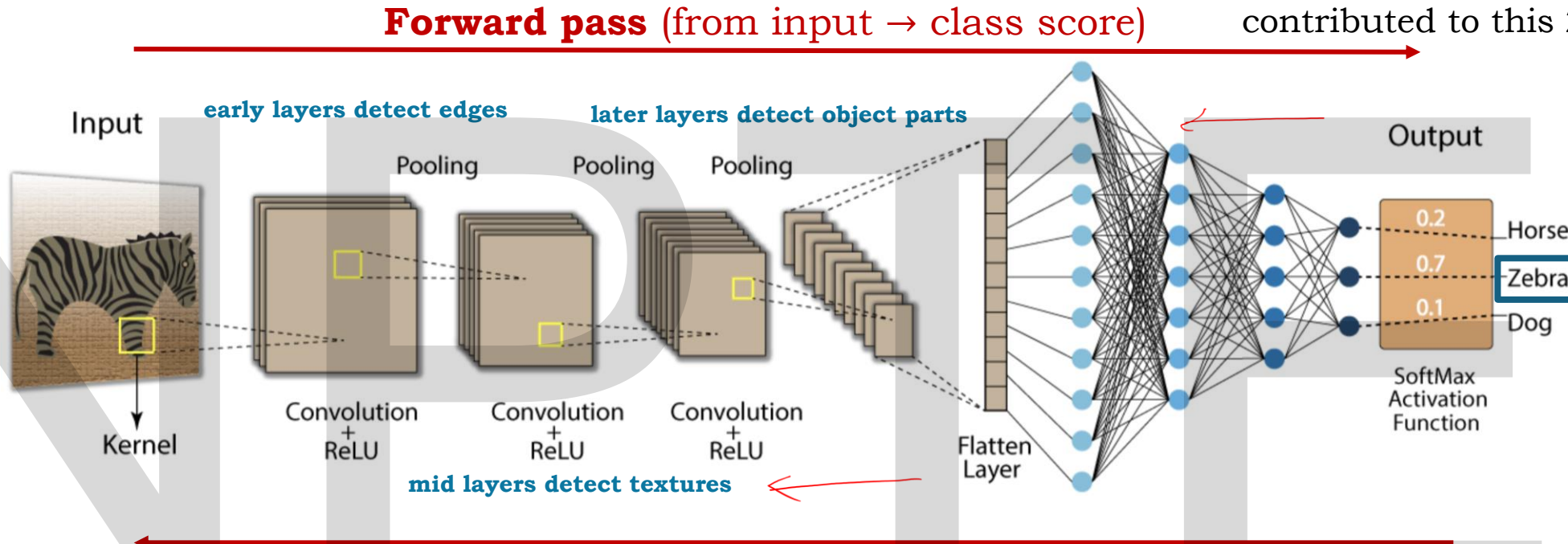
# Today's Session Overview

In today's session, we will explore:

➤ Guided backpropagation

# Guided backpropagation

Suppose the model predicts *Zebra*.  
We want to see which pixels contributed to this Zebra score.

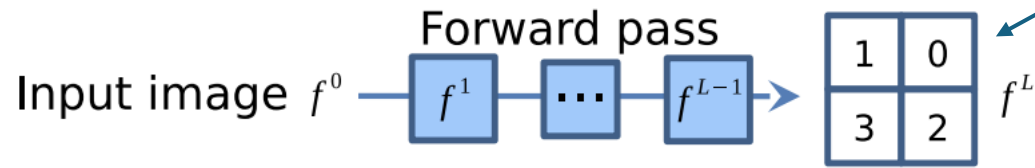


To do this, we compute the gradient of the Zebra score with respect to the activations in each layer.

**Backward pass** (take gradient of the zebra score → pass it through the fully connected layers → then through convolution + ReLU layers → finally reaching the input image pixels)

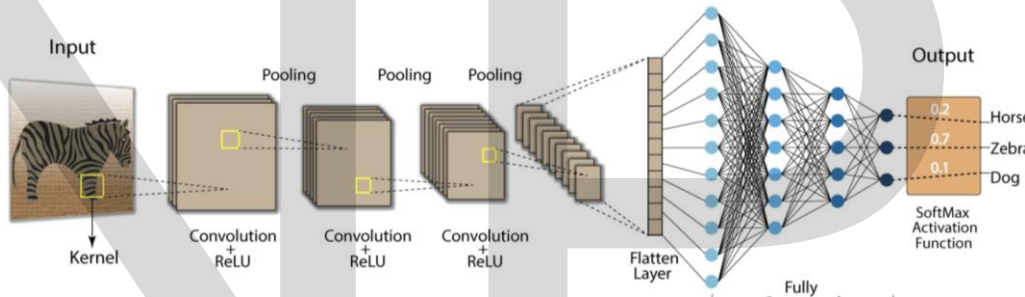
This image is passed into the CNN, where it goes through many layers — convolution, ReLU, pooling.

**Guided Backpropagation** is an XAI technique that helps us understand which pixels of the input image positively contributed the most to the model's prediction by calculating the gradients of the class score with respect to the input image.



This is the output of the last convolution + ReLU layer.

- The process starts from the **input image**, labeled as  $f^0$
- $f^1, f^2, \dots, f^{L-1}$ , represent the intermediate feature maps at each layer of the CNN.



### Backpropagate through fully connected layers:

Normal chain rule.

No Guided Backprop modification here.

**Reach the final convolution + ReLU layer:** Now we apply the Guided Backprop rules:

### Forward ReLU mask and Backward ReLU mask

Guided Backprop only modifies the **ReLU backward rule**.

So, the Guided Backprop logic starts *when the gradient reaches the first ReLU on the way backwards* — which is the last convolution + ReLU layer.

**Forward ReLU Mask:** Identify which neurons were active in the Forward Pass

Forward pass

|    |    |    |
|----|----|----|
| 1  | -1 | 5  |
| 2  | -5 | -7 |
| -3 | 2  | 4  |

Feature map after Conv layer

$\max(0, x)$   
ReLU

|   |   |   |
|---|---|---|
| 1 | 0 | 5 |
| 2 | 0 | 0 |
| 0 | 2 | 4 |

**Forward ReLU Mask:** Record of which activation at position  $i$  in layer  $l$  were active (positive) in the forward pass.

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

$(f_i^l > 0)$

During backpropagation, ReLU uses this mask to block or allow gradients.

$$\text{Derivative } f'(x) = \begin{cases} 0 & \text{for } x \leq 0 \\ 1 & \text{for } x > 0 \end{cases}$$

## Forward Pass

- The convolutional layer produces values  $z_1, z_2, \dots, z_9$ .
- ReLU transforms them:

$$z_i = \max(0, z_i)$$

- Only positive activations pass through.
- This corresponds to

|       |       |       |
|-------|-------|-------|
| $z_1$ | $z_2$ | $z_3$ |
| $z_4$ | $z_5$ | $z_6$ |
| $z_7$ | $z_8$ | $z_9$ |

These activations continue through flatten → dense → output (output is compared to the desired output → loss (error))

- During backpropagation, the algorithm calculates the gradients of the error (Zebra score with respect to EACH of the 3×3 values) in the network

$$\frac{\partial \text{Error}}{\partial z_i} \sim \frac{\partial s_{\text{zebra}}}{\partial z_i} \text{ (example)}$$

|                                              |                                              |                                              |
|----------------------------------------------|----------------------------------------------|----------------------------------------------|
| $\frac{\partial \text{Error}}{\partial z_1}$ | $\frac{\partial \text{Error}}{\partial z_2}$ | $\frac{\partial \text{Error}}{\partial z_3}$ |
| $\frac{\partial \text{Error}}{\partial z_4}$ | $\frac{\partial \text{Error}}{\partial z_5}$ | $\frac{\partial \text{Error}}{\partial z_6}$ |
| $\frac{\partial \text{Error}}{\partial z_7}$ | $\frac{\partial \text{Error}}{\partial z_8}$ | $\frac{\partial \text{Error}}{\partial z_9}$ |

- Let us assume that these are the gradient values obtained at this particular layer during backpropagation.

|    |    |    |
|----|----|----|
| -2 | 3  | -1 |
| 6  | -3 | 1  |
| 2  | -1 | 3  |

During backpropagation, ReLU uses the **forward ReLU mask** to allow gradients only through neurons that were active in the forward pass. ReLU **sets the gradients to zero** at inactive locations.

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

$$(f_i^l > 0)$$

Forward ReLU  
Mask

\*

|    |    |    |
|----|----|----|
| -2 | 3  | -1 |
| 6  | -3 | 1  |
| 2  | -1 | 3  |

$$R_i^{l+1}$$

Raw Gradients from  
Backpropagation

=

|    |    |    |
|----|----|----|
| -2 | 0  | -1 |
| 6  | 0  | 0  |
| 0  | -1 | 3  |

$$R_i^l$$

Standard  
Backprop  
Through ReLU

$$R_i^l = (f_i^l > 0) \cdot \underline{R_i^{l+1}}$$

→ standard way backpropagation flows through ReLU

Guided Backprop = Standard Backprop rule  $\times$  DeconvNet rule

Create the backward ReLU mask:

$$R_i^{l+1} > 0 =$$

|   |   |   |
|---|---|---|
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 0 | 1 |

This mask tells you:

✓ 1 = gradient is positive

→ allow

✓ 0 = gradient is negative

→ block

It only checks whether the incoming gradient is positive or negative.

➤ If the gradient is positive, DeconvNet keeps it.

➤ If the gradient is negative, it sets it to zero.

This forms a **backward-positive mask (backward ReLU mask)**.

|    |    |    |
|----|----|----|
| -2 | 3  | -1 |
| 6  | -3 | 1  |
| 2  | -1 | 3  |

This is the gradient flowing into this layer during backprop.

DeconvNet rule (Apply the mask):  $R_i^l = (R_i^{l+1} > 0) \cdot R_i^{l+1}$

If the gradient coming in is negative, block it.

If positive, allow it.

This removes negative influences and sharpens the visualization.

**DeconvNet rule**

|   |   |   |
|---|---|---|
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 0 | 1 |

|    |    |    |
|----|----|----|
| -2 | 3  | -1 |
| 6  | -3 | 1  |
| 2  | -1 | 3  |

Raw gradient

=

|   |   |   |
|---|---|---|
| 0 | 3 | 0 |
| 6 | 0 | 1 |
| 2 | 0 | 3 |

Filtered gradient

## Saliency Maps (Vanilla Gradients)

$$\text{Guided Backprop} = \boxed{\text{Standard Backprop rule}} \times \text{DeconvNet rule}$$

$$(f_i^l > 0) \cdot R_i^{l+1} \quad (R_i^{l+1} > 0) \cdot R_i^{l+1}$$

|    |    |    |
|----|----|----|
| -2 | 0  | -1 |
| 6  | 0  | 0  |
| 0  | -1 | 3  |

Standard Backprop mask (forward activations > 0) × DeconvNet mask (backward gradients > 0).

Guided Backprop = apply **both gates at the same time**:

$$R_i^l = \underbrace{(f_i^l > 0)}_{\text{active neuron}} \cdot \underbrace{(R_i^{l+1} > 0)}_{\text{positive gradient}} \cdot R_i^{l+1}$$

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 0 |
| 0 | 1 | 1 |

|   |   |   |
|---|---|---|
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 0 | 1 |

|    |    |    |
|----|----|----|
| -2 | 3  | -1 |
| 6  | -3 | 1  |
| 2  | -1 | 3  |

=

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 6 | 0 | 0 |
| 0 | 0 | 3 |

**For each location:**

- **High value** = important feature
- **Zero value** = not important
- **Guided Backprop removes all negative influences**, so only positive evidence is shown.

**Value 6:** This is the **strongest** gradient. This region **strongly supports the final class prediction**.

• Meaning the model heavily relied on this part of the image or feature

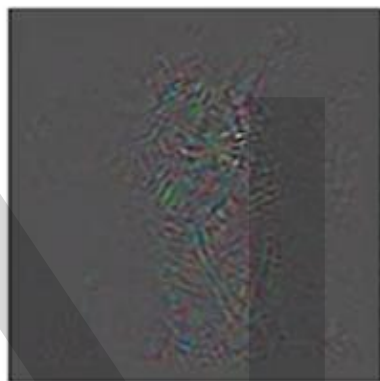
If you visualize this:

- the cell containing **6** will appear **brightest**
- the cell containing **3** will appear **less bright**
- cells with **0** will appear **dark**

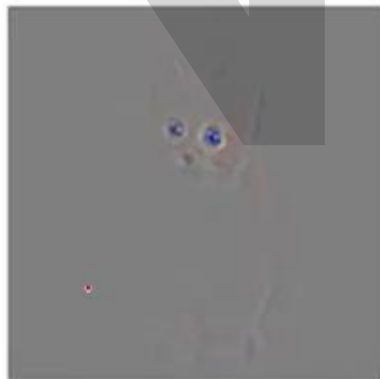
This becomes the **saliency map** (importance map).

# Guided Backpropagation

Backpropagation  
Saliency Maps (Vanilla Gradients)

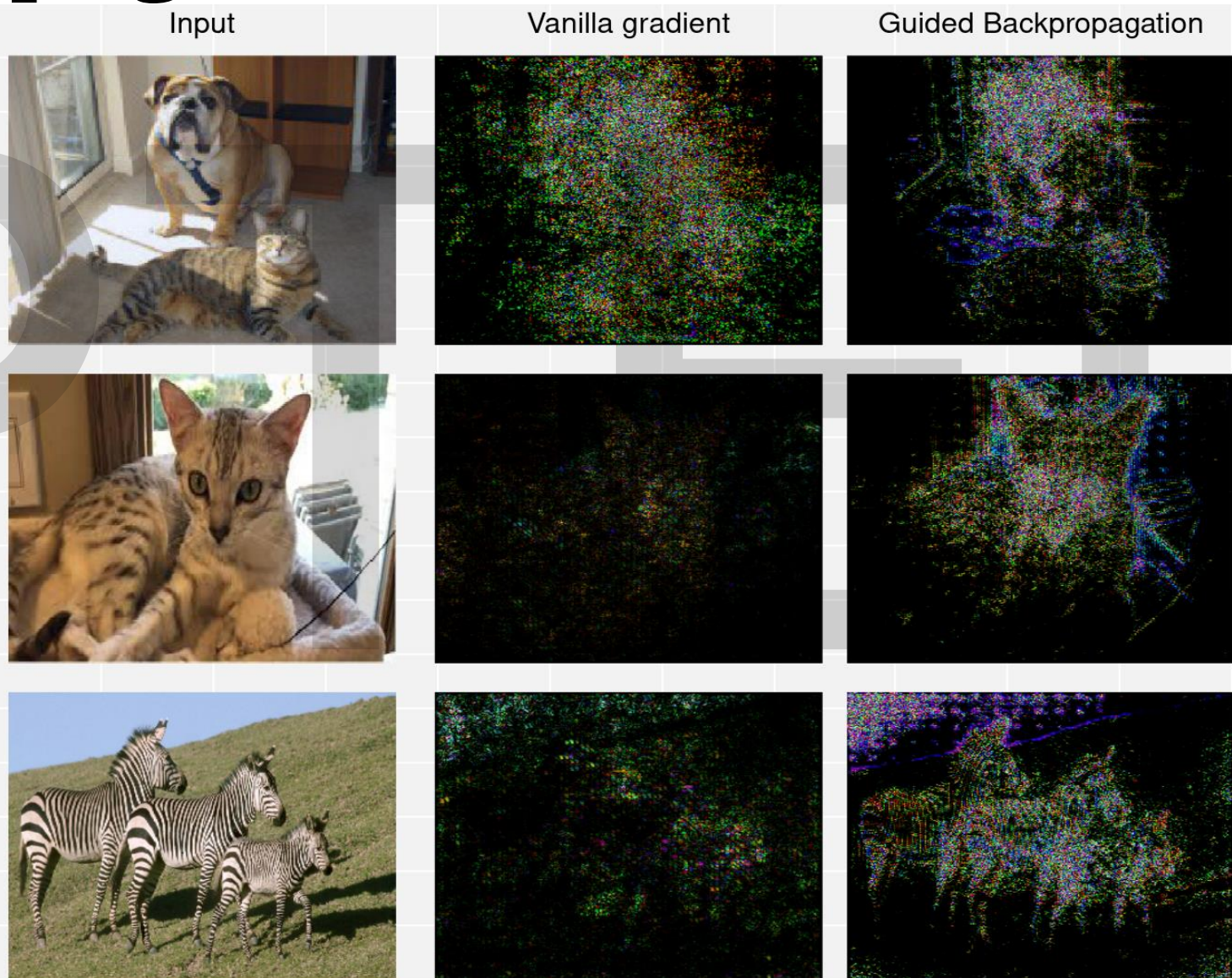


|    |    |    |
|----|----|----|
| -2 | 0  | -1 |
| 6  | 0  | 0  |
| 0  | -1 | 3  |



Guided Backpropagation

Image source: Internet



# Next Session

- Gradient-weighted Class Activation Mapping (Grad-CAM)
- Class Activation Maps (CAMs)

NIPTELL

**Thank you**